explAIn

How we got here

Almost none of the maths is new. Compute, data and one architecture changed.

"Almost none of the maths is new."

Ask on every project

1. Which part of this is actually new: the maths, the architecture, the data or the compute?
2. What is the oldest idea this depends on, and what year is it from?
3. If the maths is old, what changed recently to make it work at all?
4. Has this approach been tried and abandoned before, and what killed it then?
5. Are we promising something sized for hardware and data we do not have yet?

Rules of thumb

- Date the idea before you call it a breakthrough.
- Separate the four layers: mathematics, architecture, data, compute. Novelty sits in one.
- Training is gradient descent on a fitted curve: Cauchy 1847, at industrial scale.
- When a field over-promises, the next winter is a funding decision, not a maths one.

Red flags

- ⚠ A vendor calls a technique new and cannot name a year for it
- ⚠ A roadmap that needs compute or data nobody has yet
- ⚠ "Neural network" used as proof that something is modern
- ⚠ A claim of general intelligence with one demo behind it

Key terms

Gradient descent	Walking downhill on an error curve to find better parameters. Cauchy, 1847.
Backpropagation	The chain rule applied backwards through layers to assign blame. Published 1970.
Perceptron	Rosenblatt's 1958 learning machine: the first neural network you could train.
AI winter	A spell when funding and interest collapsed: roughly 1974–80 and 1987–93.
Transformer	The 2017 architecture that made training parallel across a whole sequence.

CHECKLIST

Date the claim in four questions

- ☐ Write down what the vendor or the paper says is new, in one sentence.
- ☐ Sort it into one of four layers: mathematics, architecture, data, compute.
- ☐ Find the oldest idea it depends on, and the year that idea was published.
- ☐ Ask what changed recently that made the old idea work: usually scale, not insight.
- ☐ If nothing changed except the name, treat it as packaging and price it that way.

explAIn

What is an AI system?

Most of what users call "the AI" is the system you build around the model.

"The model generates. The system does."

Ask on every project

1. Which ring is this really: hand-written rules, classic ML or a generative model?
2. What exactly is sent to the model on each call, and who decides that?
3. Where do facts the model can't know come from: private data, recent events?
4. Which actions can it take, and who or what approves them before they run?
5. How will we know it got worse after a model, prompt or data change?

Rules of thumb

- Name the box before you fix anything: model, context, knowledge, tools or another.
- Rule out context, knowledge and tools before you blame the model.
- Use a plain rule instead of a model when a fixed rule does the job reliably.
- Judge quality on a set of test cases you can rerun, not on one good demo.

Red flags

- ⚠ "The AI decided" and nobody can say which part made the call
- ⚠ Nobody can list what goes into the model's context on a request
- ⚠ The model can take actions with no permission check or approval step
- ⚠ A launch plan with no way to measure quality after changes

Key terms

Model	Trained weights behind an endpoint: text in, predicted text out.
Inference	Running the trained model on a request. Its weights don't change.
Orchestration	The logic that decides which steps, model calls and tools run, and in what order.
Guardrails	Checks and limits around the model: filters, permissions, approvals.
Evaluation	Repeatable tests that show whether the system is good enough, and stays so.

CHECKLIST

System map in five minutes

- ☐ Pick one real user request and trace it from the interface to the answer, step by step.
- ☐ Mark the model call. Every other step belongs to the system.
- ☐ Sort each step into the eight boxes; one step can touch two boxes.
- ☐ For each empty box, ask: missing by design, or missing by accident?
- ☐ Note what each box adds per request in cost and delay.
- ☐ Circle the box most likely to fail first, and name who owns it.

explAIn

How a model sees the world

Every answer is shaped by what's in the context: choose it, measure it, pay for it.

"Context is the model's working reality."

Ask on every project

1. What is in the context on a typical call, and how many tokens does each part take?
2. Which parts of that context does this answer actually need?
3. Where do the key rule and facts sit: at the start, buried mid-way, or by the question?
4. What gets trimmed when the window fills, and would anyone notice?
5. Do any sources in the context contradict each other or include outdated versions?

Rules of thumb

- Count tokens with the model's own tokenizer; don't estimate from word counts.
- Send only what this request needs; cut stale versions and unrelated history.
- Keep critical rules in instructions sent every call; restate them near the question.
- Estimate tokens per call × calls per day before deciding how much context to send.

Red flags

- ⚠ "Just put the whole handbook in the prompt, the window is big enough"
- ⚠ Old and current versions of a policy side by side in the same context
- ⚠ Nobody knows the token count or cost of a typical request
- ⚠ Testing only on short chats, never on long conversations

Key terms

Token	A piece of text (word, word part or symbol) that the model reads as one number.
Embedding	A long list of numbers representing a token; related meanings sit close together.
Attention	How each token weighs earlier tokens to pull in what's relevant to it.
Context window	The hard token limit for everything in one call, the answer included.
Dilution	Relevant text used less reliably because unrelated text competes with it.

CHECKLIST

Context budget check

- ☐ List every source in the context: instructions, history, documents, tool output, question.
- ☐ Count tokens per source with the real tokenizer and note the biggest share.
- ☐ Cut what this request doesn't need: stale versions, unrelated turns, noisy tool output.
- ☐ Keep key rules in instructions, label key facts, and put the question near the end.
- ☐ Check what gets trimmed near the window limit, and leave room for the answer.
- ☐ Multiply tokens per call by calls per day, and time a long request end to end.

explAIn

How models learn

Weights change only in training. Every other change lives in context, memory or state.

"The model keeps capacity, not records."

Ask on every project

1. Does this need new behavior from the model, or just new facts in the context?
2. How often does this information change, and who must be able to update it?
3. Should this change reach one request, one user, one tenant or everyone?
4. Is this fact frequent, rare, private or newer than the training cutoff?
5. When the provider ships a new version, what do we have to re-test or redo?

Rules of thumb

- Keep facts that change in a source of truth fed into context, never in the weights.
- Consider fine-tuning for consistent style, format or a narrow task, not to add facts.
- Never read a wrong answer to judge whether it knew. Check a source you control.
- Treat user feedback as logged data: it changes nothing until someone runs training.

Red flags

- ⚠ "Users will correct it and it'll learn over time", with no pipeline behind it
- ⚠ A plan to fine-tune on docs so the model "knows" current policies
- ⚠ A feature that treats the model's own "I'm not sure" as a signal
- ⚠ A provider version upgrade with no re-test of prompts, formats and refusals

Key terms

Parameters	The numbers (weights) set by training. They don't move while the model is used.
Post-training	Further training that turns a text predictor into an assistant with set behaviors.
Overfitting	Memorising the training data instead of learning patterns that transfer.
Memory	Stored notes or history that the system re-sends as context on later requests.
Application state	What your product writes (orders, files, logs); it reaches the model only if sent.

DECISION GUIDE

Where does this change live?

If it matters for this request only	→ **put it in the context.**
If one user needs it across sessions	→ **save it as memory, re-sent as context.**
If it's a fact that changes, like a price or policy	→ **fix the source of truth and retrieve it.**
If it records what happened, like an order or feedback	→ **it's application state.**
If you need behavior that prompts can't get	→ **consider a fine-tune, then evaluate it.**
If the provider ships a new version	→ **your parameters moved; re-test prompts and formats.**

explAIn

Reasoning, sampling and hallucination

Treat every output as a likely guess until something outside the model checks it.

"Fluency is not the same as truth."

Ask on every project

1. What does one wrong output cost here, and who pays for it?
2. Which parts are exact values: numbers, names, dates, IDs or citations?
3. What catches an error before a user sees it, and does it depend on the model?
4. Where do we need the same output twice, for tests, audits or caching?
5. Would more reasoning help, or is the model missing a fact nobody gave it?

Rules of thumb

- Get numbers, prices and policy values from source systems; let the model only phrase them.
- Store outputs you must reproduce; never count on rerunning the prompt to get them back.
- Tune temperature for variety, not accuracy: a low setting can repeat a wrong answer.
- Spend reasoning on math, code and planning; for missing facts, add sources, not thinking.

Red flags

- ⚠ The only verification step is asking the model to double-check itself
- ⚠ Citations, IDs or amounts in the output with no lookup behind them
- ⚠ Quality judged from one good demo run, with no repeats and no test set
- ⚠ "A human reviews it" with no time, tooling or sample rate to make it real

Key terms

Token probability	The score the model gives each possible next token before one is picked.
Temperature	A setting that sharpens or flattens those odds before a token is sampled.
Hallucination	Fluent output that isn't supported by facts or by the sources provided.
Test-time compute	Extra tokens spent reasoning at answer time: better odds, more cost and latency.
Deterministic check	A check in code, data or rules that doesn't depend on the model's own guess.

DECISION GUIDE

Probabilistic or verified?

If variety is the point (names, drafts, ideas)	→ **leave it probabilistic; test with real users**
If a person reads it next to the source	→ **probabilistic is OK; show or link the source**
If code consumes it	→ **validate the schema and the values; reject, retry or fall back**
If it states a number, price, date or citation	→ **fetch it from a source system; don't generate it**
If it triggers an action or moves money	→ **code computes and permits it; a person approves risky ones**
If an error can't be caught or undone	→ **don't automate it; keep a person deciding**

explAIn

Context engineering

Treat the context like an API contract: every part needs an owner and a trust level.

"Construct the world the model reasons inside."

Ask on every project

1. What exactly does the model receive on one request, in order? Can we log it?
2. Who owns each part of the context, and when was it last reviewed?
3. Which parts can a user, an email, a web page or a tool result write into?
4. If injected text took control, what is the worst the system would let it do?
5. What gets trimmed, summarized or dropped when the window fills up?

Rules of thumb

- Fix a conflicting instruction at its source; never add a rule to overrule another.
- Label untrusted content with its source and pass it as data, never as instructions.
- Vary few-shot examples and include an edge case; the model copies their flaws too.
- Cap what a hijacked model can do: least-privilege tools, approval for sends and deletes.

Red flags

- ⚠ Nobody can say who owns the system prompt
- ⚠ The prompt grows by one rule per incident and nothing is ever removed
- ⚠ Retrieved text or tool output pasted in raw, with no labels or delimiters
- ⚠ One flow mixes private data, untrusted content and a way to send data out

Key terms

Data leakage	Private data reaching a prompt, a log or a reply that should not hold it.
System instructions	Your team's role, rules and format, re-sent every call; weighted higher, not enforced.
Few-shot examples	Sample inputs and outputs in the context that the model copies closely.
Context pollution	Stale, conflicting or irrelevant text that distracts or misleads the model.
Indirect injection	Instructions hidden in content the model reads, like an email, page or tool result.

TEMPLATE

Context assembly template

Role and rules: purpose, hard limits and tone; one owner, no conflicts, stable so it caches

Examples: a few varied input and output pairs in the target format, including an edge case

Reference material: only relevant chunks, delimited, labeled with source and date, as data

Tool results: the fields the task needs, not raw dumps, labeled with the system they came from

History: recent turns that still matter; older ones summarized or dropped

Task and output format: the current question restated, plus the schema or format required

explAIn

Retrieval, memory and fine-tuning

Before choosing a mechanism, ask: is this information, state or behavior?

"Information, state and behavior are different problems."

Ask on every project

1. Is this information to know, state to track, or behavior to shape?
2. How often does this knowledge change, and who updates the source?
3. Must the answer be exact and live, or is the closest match good enough?
4. Who may see each document or memory, and is that filtered before retrieval?
5. When an answer is wrong, can we see which chunks were retrieved?

Rules of thumb

- Retrieve facts that change or need a citation; don't bake them into the weights.
- Read orders, balances and permissions from the database of record, never from memory.
- Combine meaning-based and keyword search when users look up codes, names or IDs.
- Fine-tune only after prompting falls short and you have labeled examples and an eval set.

Red flags

- ⚠ Fine-tuning proposed to teach the model the company's documents
- ⚠ A wrong answer blamed on the model before anyone checked what was retrieved
- ⚠ Access permissions applied after retrieval, or not at all
- ⚠ Stored memory that users can't see, correct or delete

Key terms

LoRA	A cheap fine-tune: train small added weights, leave the base model alone.
Chunking	Splitting documents into passages that can be found and read on their own.
RAG	Retrieval-augmented generation: search first, then answer from what was found.
Application memory	State the system stores and re-sends, like preferences or summaries of past chats.
Fine-tuning	Further training on your examples; it changes the weights, and so the behavior.

DECISION GUIDE

Pick the mechanism

If it's a stable rule, tone or short style guide	→ **prompt, with good and bad examples**
If it's documents that change or need sources	→ **retrieval with citations and metadata filters**
If it's per-user preferences or context across sessions	→ **application memory the user can see**
If it must be exact and live (orders, balances, permissions)	→ **database, via code or a tool**
If prompts can't hold a format or narrow task at your volume and cost	→ **fine-tune, with evals**
If it spans several	→ **combine them, but name the primary mechanism and who owns it**

explAIn

How models interact with the real world

The model only asks. Your code decides what runs, and with whose rights.

"The model requests. Another system acts."

Ask on every project

1. What is the worst thing each tool can do if the model calls it wrongly?
2. Whose credentials does each tool run with: the user's, or a service account?
3. Which actions can't be undone, and where does the user confirm them?
4. What happens if the same tool call is retried or sent twice?
5. Which tool results carry untrusted text, such as web pages, emails or documents?

Rules of thumb

- Give each tool one narrow job; never expose raw SQL, HTTP or shell to the model.
- Enforce limits in code; a rule that lives only in the prompt is not a control.
- Require confirmation in the interface for anything that spends, sends or deletes.
- Run every tool with the signed-in user's permissions, never an admin account.

Red flags

- ⚠ One tool like run_sql or call_api that can do almost anything
- ⚠ "We told the model not to" offered as the safety plan
- ⚠ API keys, tokens or card numbers appear in prompts or tool arguments
- ⚠ Nobody can say which tool calls ran yesterday, with which arguments

Key terms

Tool call	A structured request from the model: a tool name plus arguments your app may run.
Least privilege	Give each tool only the access its task needs, and nothing more.
Idempotency key	An ID sent with a request so a retry doesn't repeat the action, like a double booking.
Schema validation	Checking tool arguments against declared types and ranges before running.
MCP server	A program that exposes tools and data to apps through a shared, open protocol.

CHECKLIST

Tool design review

- ☐ One narrow job, with a name and description clear enough for the model to choose it correctly
- ☐ Risk level set: read, reversible write, irreversible, or never exposed to the model
- ☐ Arguments typed and validated in code: types, enums, ranges, and that IDs belong to the user
- ☐ Irreversible or costly actions need user confirmation in the interface, not in chat text
- ☐ Runs with the user's scoped permissions; credentials stay in the app, never in context
- ☐ Every call logged with arguments and result; retries carry an idempotency key

explAIn

Agents without the mythology

Autonomy is a cost: give the model a decision only where code can't cover the cases.

"An agent is a loop with model-selected decisions."

Ask on every project

1. Who decides the next step here: our code, or the model?
2. Are the steps the same for every input, or do they depend on what it finds?
3. What stops the loop: a step limit, a budget, a timeout or a human?
4. If step 3 is wrong, which later step or person notices it, and when?
5. If a run fails halfway, what state do we need to resume it or undo it?

Rules of thumb

- Make known steps fixed code; keep model decisions for the parts that truly vary.
- Set step, budget and time limits in code, and decide what happens when one is hit.
- Check key results before they enter state; later steps will treat them as facts.
- Put approval on few, high-risk actions, and show the approver the evidence.

Red flags

- ⚠ It's called an agent, but nobody can say what the model decides
- ⚠ No step limit or budget, so a stuck run keeps retrying the same action
- ⚠ One person approves hundreds of actions a day without reading them
- ⚠ More agents are added before a simple workflow was tried and measured

Key terms

Workflow	Code fixes the order of steps; the model only fills in some of them.
Agent loop	The model picks the next action, the system runs it, and the result feeds the next turn.
Router	One model decision that sends an input down one of several fixed paths.
Retry budget	A hard cap on repeated attempts, so a failing step cannot consume the run.
Blast radius	How much damage one wrong action can do before anyone can stop or undo it.

DECISION GUIDE

How much autonomy?

If the steps are the same for every input	→ **fixed workflow; the model only fills in steps**
If inputs fall into a few known paths	→ **router: one model decision, then fixed code**
If the path depends on what it finds and tools are read-only	→ **bounded agent with caps**
If an action sends, spends, deletes or changes production	→ **a human approves it first**
If an action can't be undone and approval isn't practical	→ **don't expose it as a tool**
If you can't yet measure whole runs	→ **stay one level less autonomous until you can**

explAIn

Building AI you can trust

If you can't name the number that would make you roll back, you don't know it works.

"A demo is not a system."

Ask on every project

1. Which written cases define "working" for this feature, and who agreed on them?
2. Which quality checks can plain code do before we reach for a model judge?
3. Which failure type is most common, and what does each type cost us?
4. What reruns the set when a prompt, model, index or tool changes?
5. What p95 latency and cost per request can this use case afford?

Rules of thumb

- Grade with code first, a calibrated judge next, and people to set the rubric.
- Turn every production failure into a golden-set case before you fix it.
- Compare pass rates over repeated runs, not single outputs, before judging a change.
- Block a release when golden-set scores drop below the stored baseline.

Red flags

- ⚠ Quality is shown only with hand-picked demo examples
- ⚠ A single accuracy number with no breakdown by failure type
- ⚠ A model judge in use with no check against human labels
- ⚠ The app calls an unpinned model name and nothing reruns the set

Key terms

Golden set	A small, curated, versioned set of real cases the team agrees defines "working".
Precision and recall	How much of what it returned was right, and how much of the right it found.
LLM-as-a-judge	A model that grades outputs against a written rubric; calibrate it against people.
Regression test	Rerunning the set after a change to catch cases that used to pass and now fail.
p95 latency	The response time 95% of requests beat; it shows what the slow cases feel like.

TEMPLATE

One-page evaluation plan

Scope: the feature, its users, and what a costly error looks like

Golden set: sources (logs, tickets, past failures), size, hard cases, who maintains it

Graders: one per criterion (code check, calibrated judge or human), with the rubric

Pass thresholds: per criterion and per failure type, plus the baseline to compare against

Regression trigger: which changes rerun the set, and what score drop blocks a release

Budgets and owner: cost per request, p95 latency, and who acts when a number slips